

中山大学网络空间安全学院必修课程

第2节 常量变量

C Programming

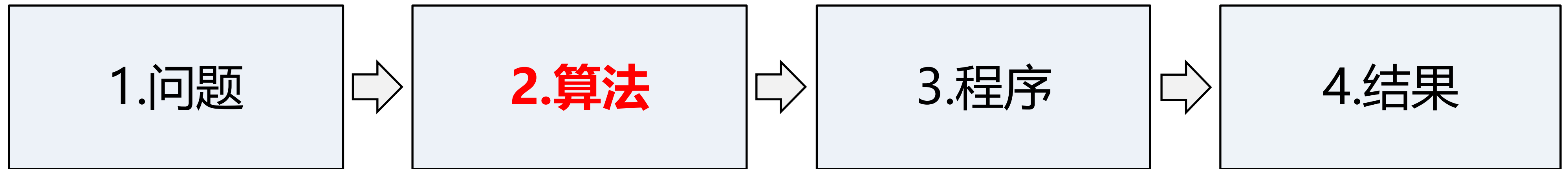
岳晟

网络空间安全学院 工学园2栋508

yuesh5@mail.sysu.edu.cn

2026年秋季

写代码之前，先要知道解决问题的方法和步骤



算法——程序的灵魂

Algorithms + Data Structures = Programs

算法 + 数据结构 = 程序

—— *Niklaus Wirth*

算法与数据结构

长方形的周长与面积

我的题目



岳晟

题面

评测

作答

统计

设置

题目描述

题目描述

本题要求读入长方形的长和宽，计算并输出长方形的周长和面积。

输入格式:

在一行中输入两个正整数 `length` 和 `width`，分别表示长方形的长和宽。

[C]

```
1  #include <stdio.h>
2
3  int main(void)
4  {
5      int length, width;
6      int perimeter, area;
7
8      scanf("%d %d", &length, &width);
9
10     perimeter = 2 * (length + width);
11     area = length * width;
12
13     printf("%d %d\n", perimeter, area);
14
15     return 0;
16 }
```

数据

算法

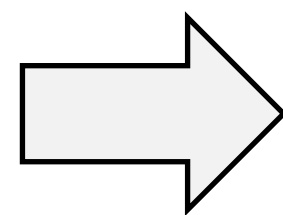
算法

- ❑ 算法：是计算机为解决一个问题而采取的方法和步骤
- ❑ 同一个问题可以有不同的算法，好的算法能极大提升**解决问题的效率**

例1 求
 $1 \times 2 \times 3 \times \dots \times 1000$

逐项相乘

先求1乘以2，得到结果2
再求2乘以3，得到结果6
再求6乘以6，得到结果24
.....



累乘

设置乘积变量 $p \leftarrow 1$ 和计数变量 $i \leftarrow 1$
 $p \leftarrow p \times i$
 $i \leftarrow i + 1$
直到 i 为1000

步骤固定
改变范围需重写大量乘法步骤

表达简洁
具备更好的可拓展性

算法

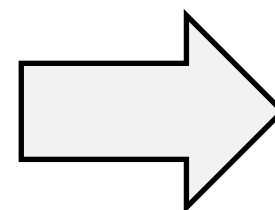
- ❑ **算法**：是计算机为解决一个问题而采取的方法和步骤
- ❑ 同一个问题可以有不同的算法，好的算法能极大提升**解决问题的效率**

例2 判断一个大于3的整数 n 是否为素数

逐个试除

从 2 试除到 $n - 1$
如果可以整除，则 n 不是素数

最多需 $n - 2$ 次计算



优化方法

从 2 试除到 $\sqrt{n}$
如果可以整除，则 n 不是素数

最多需 $\sqrt{n}$ 次计算

二十世纪十大算法

年代	算法	核心作用	典型应用场景
1946	蒙特卡洛方法	利用随机采样求解复杂问题	金融风险分析、物理模拟、概率计算
1947	单纯形法	求解线性规划与最优化问题	生产计划、资源分配、物流调度
1950	Krylov 子空间迭代法	求解大型线性方程组	工程仿真、有限元计算、大型科学计算
1951	矩阵分解方法	将复杂矩阵问题分解简化	数据分析、方程求解、科学计算
1957	FORTRAN 优化编译器	将高级语言转换为高效机器代码	科学计算、工程程序、高性能计算
1959	QR 算法	计算矩阵的特征值和特征向量	工程计算、数据分析、控制系统
1962	快速排序	高效地对大量数据排序	数据库排序、搜索预处理、软件开发
1965	快速傅里叶变换	快速进行频域分析	音频处理、图像处理、无线通信
1977	整数关系检测算法	发现实数之间的整数关系	数学研究、常数识别、符号计算
1987	快速多极子方法	加速大规模粒子相互作用计算	天体模拟、分子动力学、电磁仿真

回顾上节

□ 如何在屏幕打印文字？

```
#include <stdio.h>

int main(void){
    printf("Hello, world\n");
    return 0;
}
```

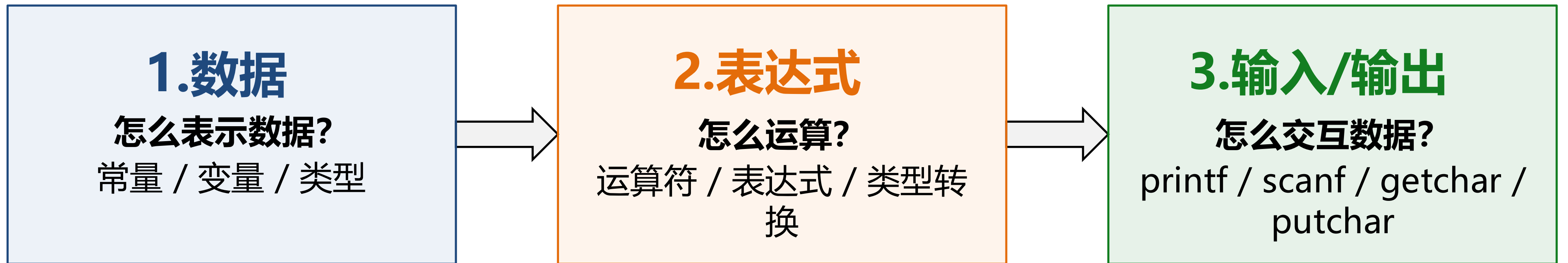
本节目标

□ 如何写出能处理数据的程序？

□ 例：求 $ax^2 + bx + c = 0$ 的根， a 、 b 、 c 由键盘输入， $b^2 - 4ac > 0$

本节内容

□ 如何写出能处理数据的程序？



求 $ax^2 + bx + c = 0$ 的根

```
#include <stdio.h>
#include <math.h>

int main(void){
    float a, b, c;
    float disc, x1, x2, p, q;

    scanf("%f%f%f", &a, &b, &c);

    disc = b * b - 4 * a * c;
    p = -b / (2.0 * a);
    q = sqrt(disc) / (2.0 * a);
    x1 = p + q;
    x2 = p - q;

    printf("x1=%7.2f\nx2=%7.2f\n", x1, x2);
}
```

数据

表达式

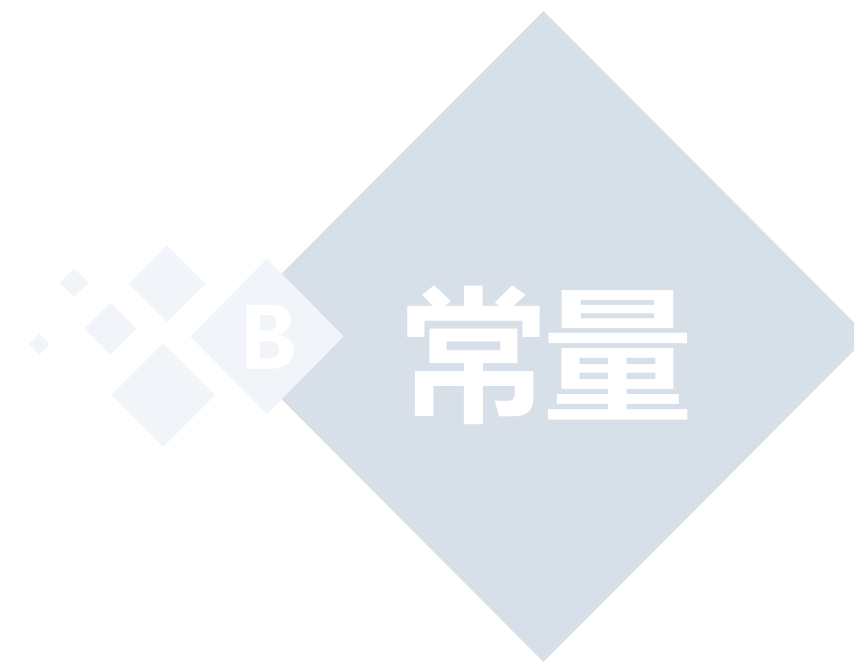
输入输出

本节内容

□ 如何写出能处理数据的程序？



在计算机高级语言中，数据的两种表现形式

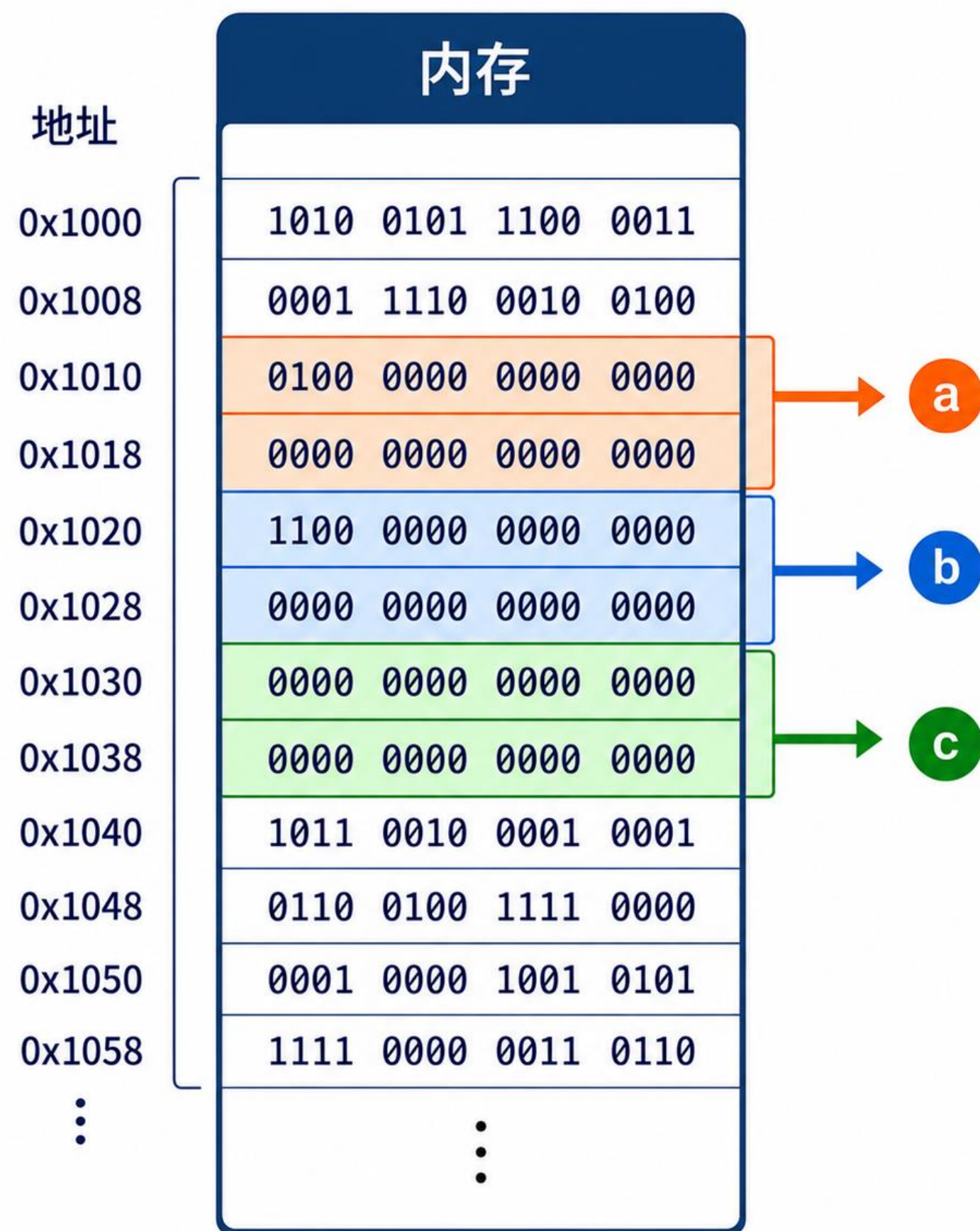


变量

- ❑ 在数学中，变量是用于指代特定值的符号，如： $y = 2x + 3$ ， y 与 x 表示可变化的值
- ❑ 在C语言中，**变量**表示储存数据的**储存单元名称**，如：

float a, b, c;
数据类型 变量名

- ❑ 变量的值在**程序运行期间可以被改变**
- ❑ C语言中变量必须**先定义、后使用**



变量的三要素

变量定义：

```
float a = 1.0 ;
```

数据类型

占用空间大小、数据解释

标识符

储存单元的名字

值

可在运行期间改变

标识符

❑ 标识符是C语言变量等对象的名称，由大小写字母、数字或下划线组成

- 合法的标识符必须以非数字字符开始

- 标识符大小写有别

- 不能使用C语言关键字

❑ 以下哪些标识符是合法的？

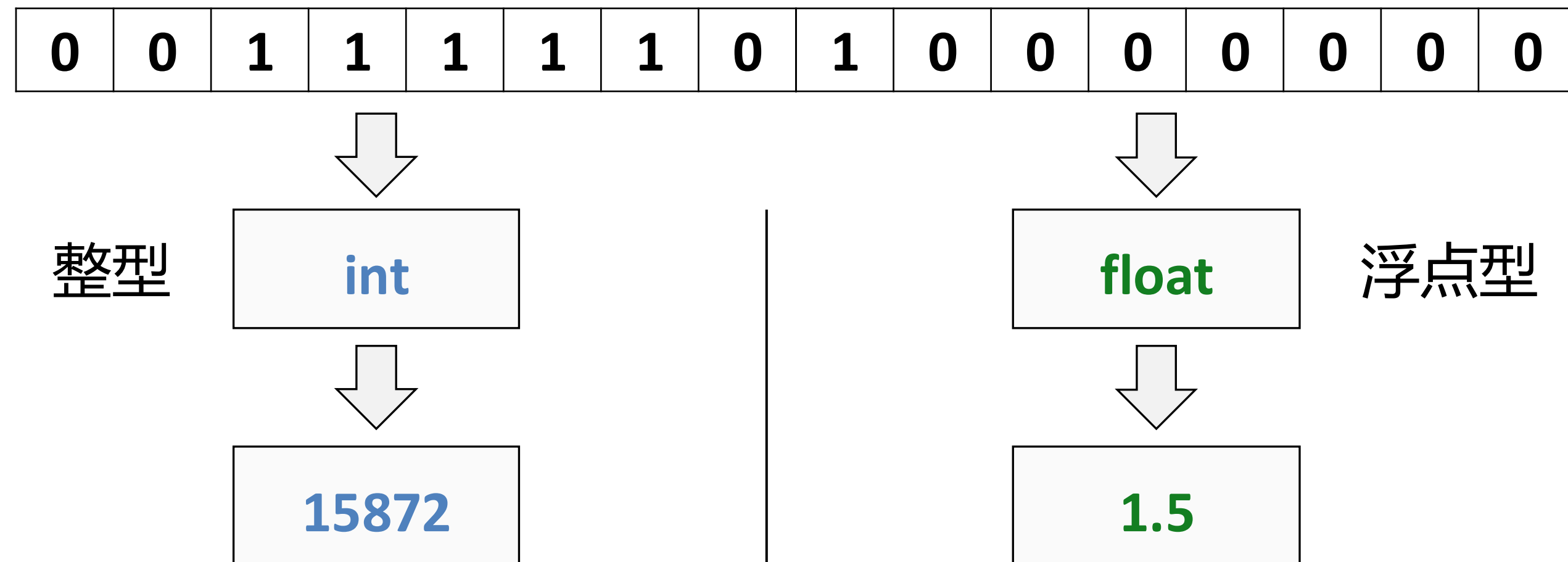
age_of_dog	✓	taxRateY2K	✓	PrintHeading	✓	Age#	✗	maxValue	✓
_EAX	✓	age-of-cat		printHeading	✗	2000TaxRate	✓	float	✗

C语言中的关键字

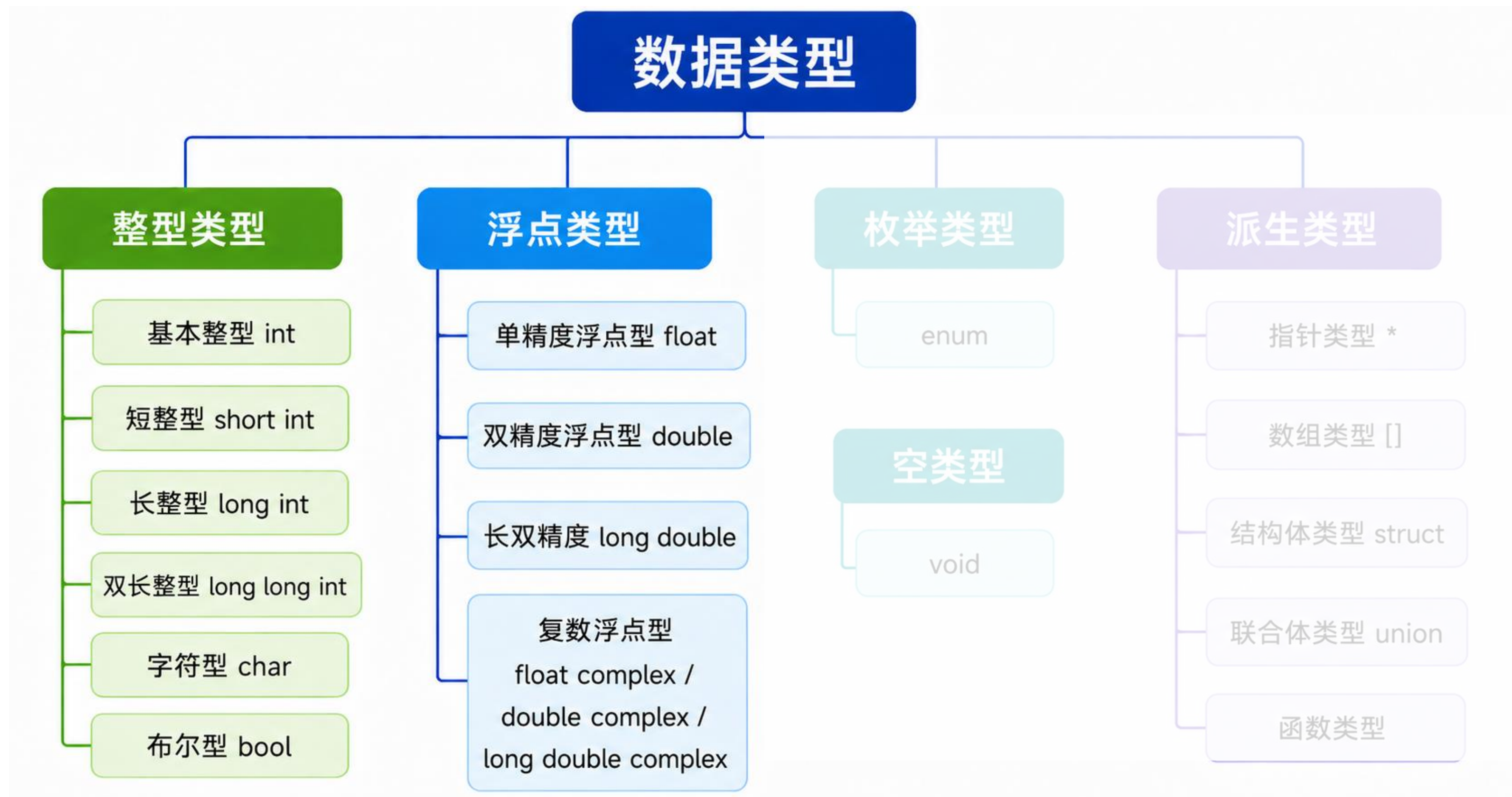
auto	break	case	char
const	continue	default	do
double	else	enum	extern
float	for	goto	if
int	long	register	return
short	signed	sizeof	static
struct	switch	typedef	union
unsigned	void	volatile	while

数据类型

- ❑ 数据类型是对数据分配存储单元的安排，包括存储单元的长度（占多少字节）以及数据的存储形式
- ❑ 告诉计算机数据需要多少储存空间、怎样解释



数据类型



整型

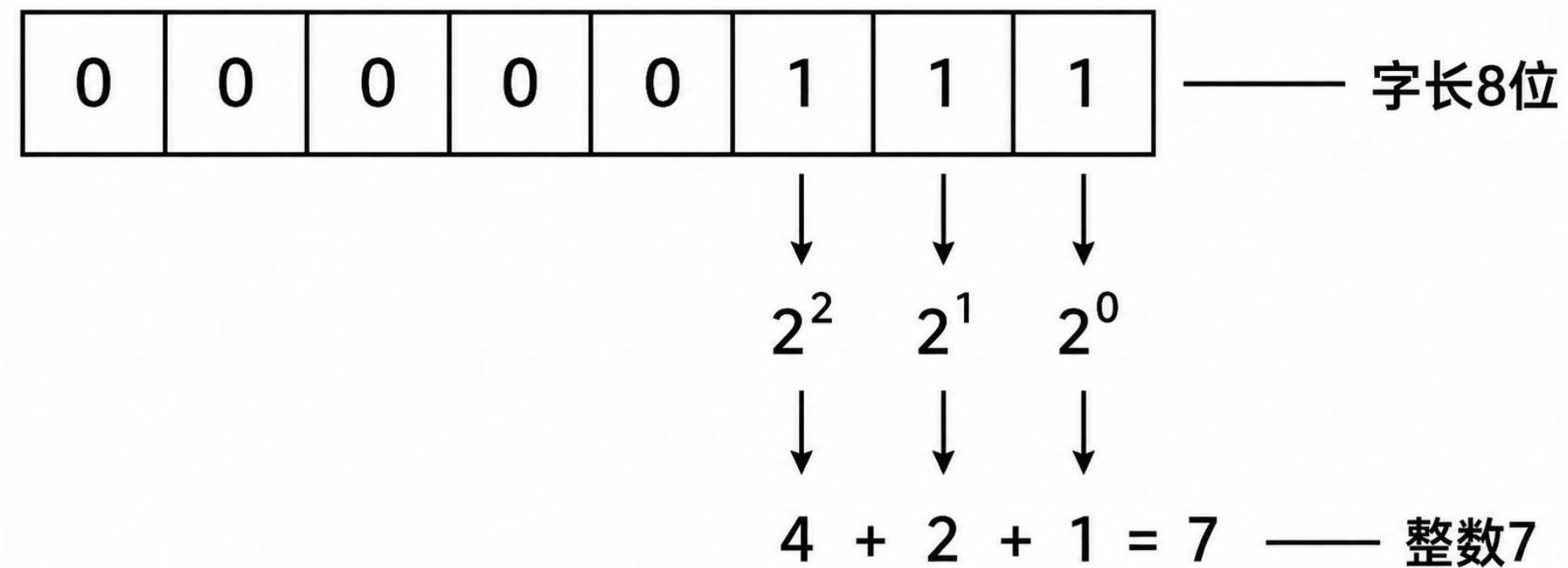
- ❑ **整数类型**：不含小数点和指数的数，如 1、-10、525
- ❑ 不同的整型可以储存不同**符号**、**大小**的整数值

?

整型数据类型	缺省形式	格式声明符	常见字节数	取值范围
signed int	int (最常用)	%d	4	$-2^{31} \sim 2^{31}-1$
unsigned int	unsigned	%u	4	$0 \sim 2^{32}-1$
signed short int	short	%hd	2	$-2^{15} \sim 2^{15}-1$
unsigned short int	unsigned short	%hu	2	$0 \sim 2^{16}-1$
signed long int	long	%ld	4	$-2^{31} \sim 2^{31}-1$)
unsigned long int	unsigned long	%lu	4	$0 \sim 2^{32}-1$
signed long long int	long long	%lld	8	$-2^{63} \sim 2^{63}-1$
unsigned long long int	unsigned long long	%llu	8	$0 \sim 2^{64}-1$

整数的存储方式

- 整数在计算机内部以二进制来存储

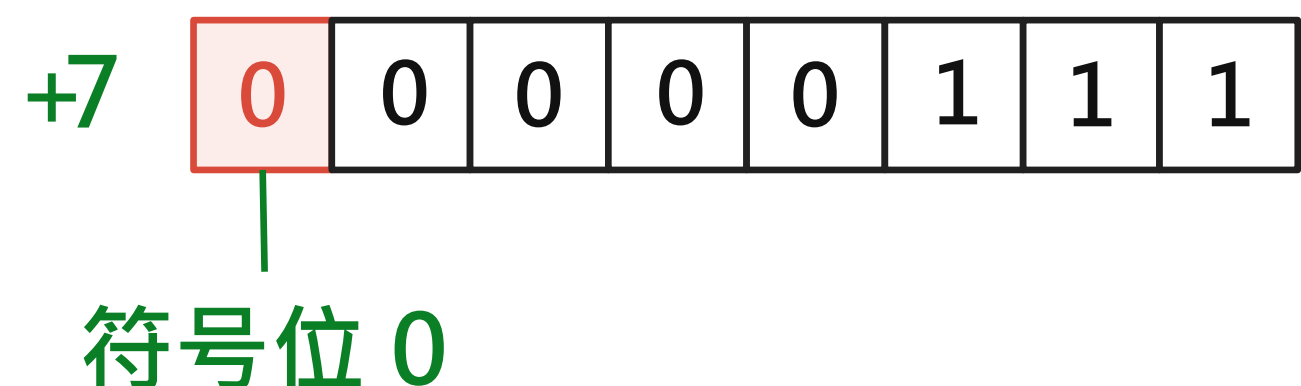


- 不同整数类型储存整型变量所用的**二进制位数**不同

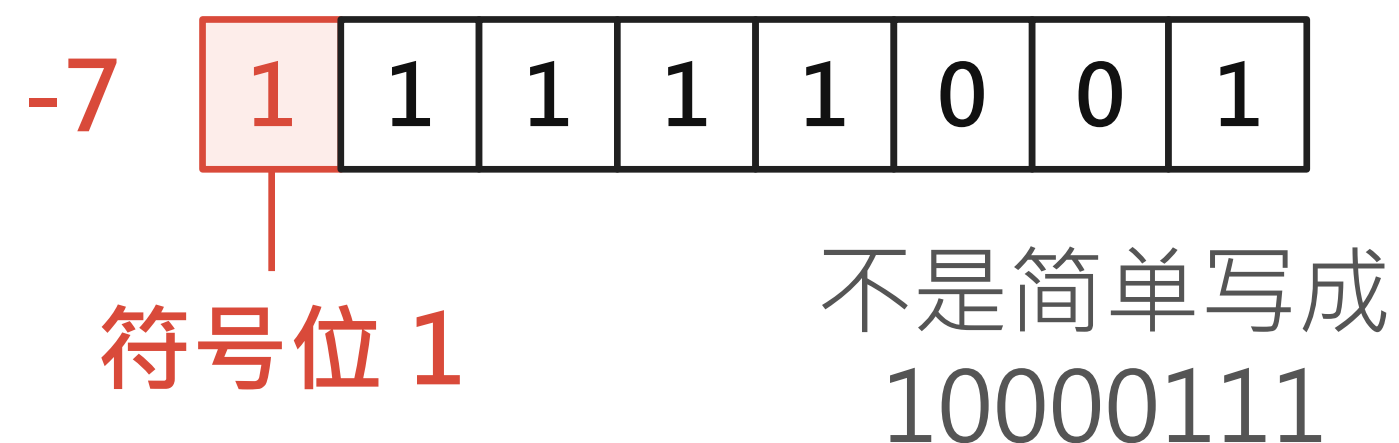
整数的存储方式

□ 有符号整数用其二进制**补码**存储，**最高位**表示符号

■ 正数的补码是其本身



■ 负数的补码为其**绝对值**的二进制编码**按位取反后加一**



整数的存储方式

❑ 负数的补码为其绝对值的二进制编码按位取反后加一

+7	0	0	0	0	0	1	1	1
-7	1	1	1	1	1	0	0	1

11111111	--	$2^8 - 1$
- 00000111	--	7求二进制
11111000	--	按位求反
+ 1	--	加一
11111001	--	-7的补码

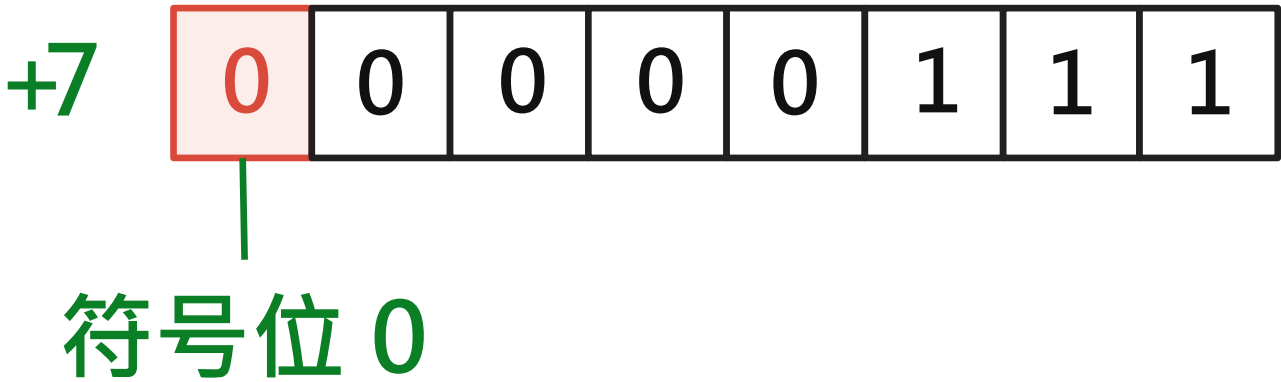
Why? 可以用加法电路实现减法

00000111	--	+7
+ 11111001	--	-7
100000000	--	低八位: 0

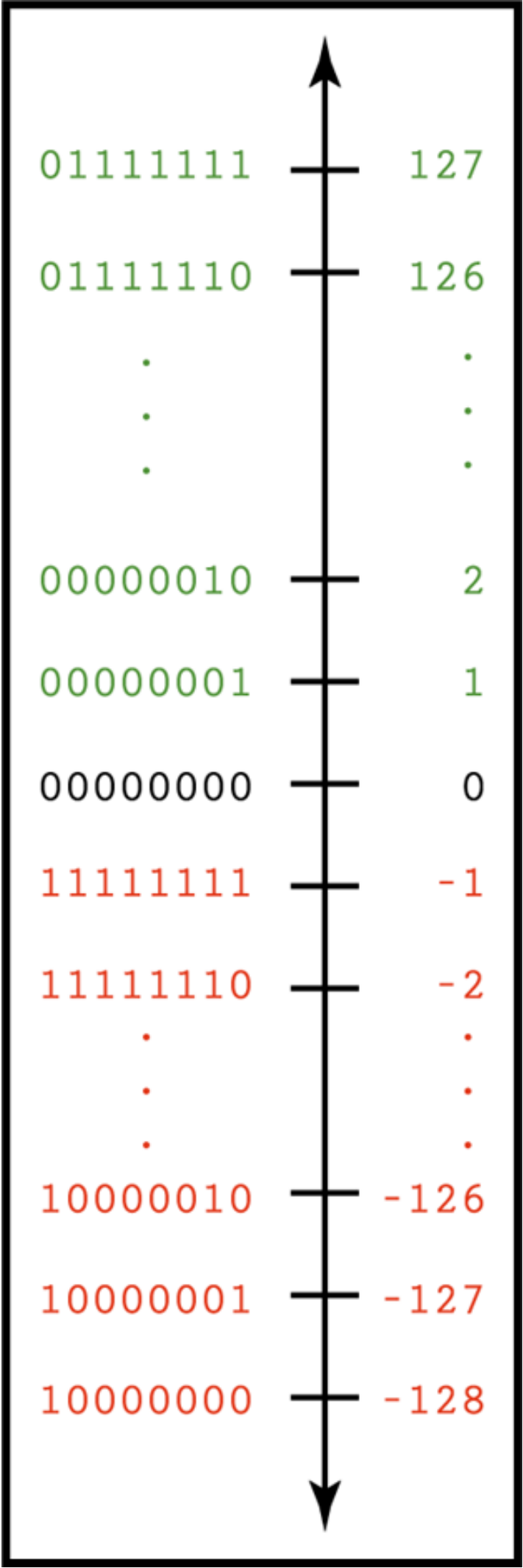
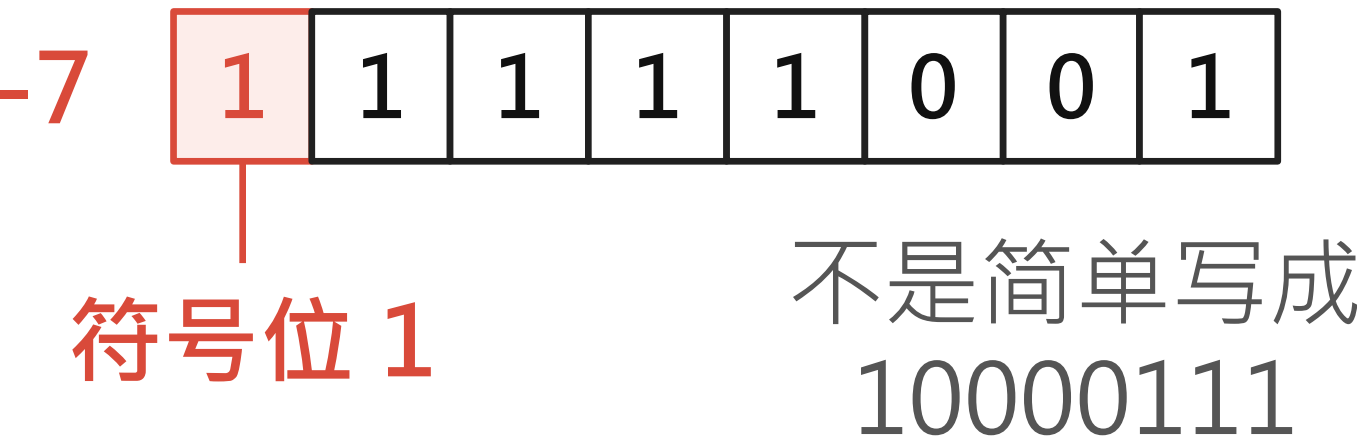
整数的存储方式

□ 有符号整数用其二进制补码存储，最高位表示符号

■ 正数的补码是其本身



■ 负数的补码为其绝对值的二进制编码按位取反后加一



练习

- 已知某负整数的 8 位补码为 11010110，求其对应正数的二进制表示及十进制值
- 00101010, 42

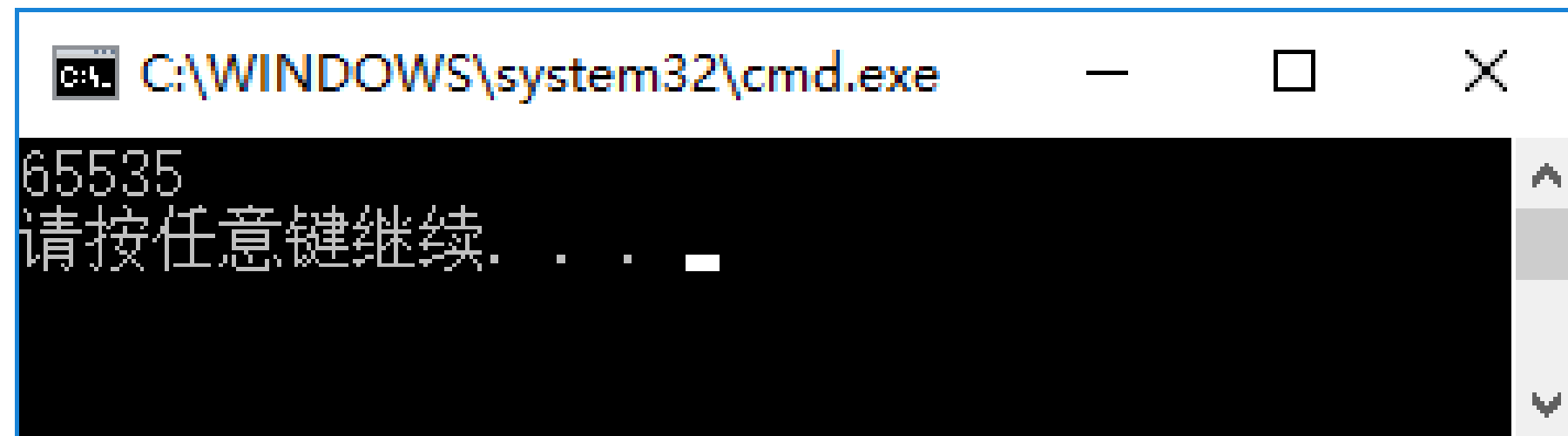
思考

□ 运行以下代码会输出什么？

```
unsigned short price = -1;  
printf("%hu\n", price);
```

price

1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---



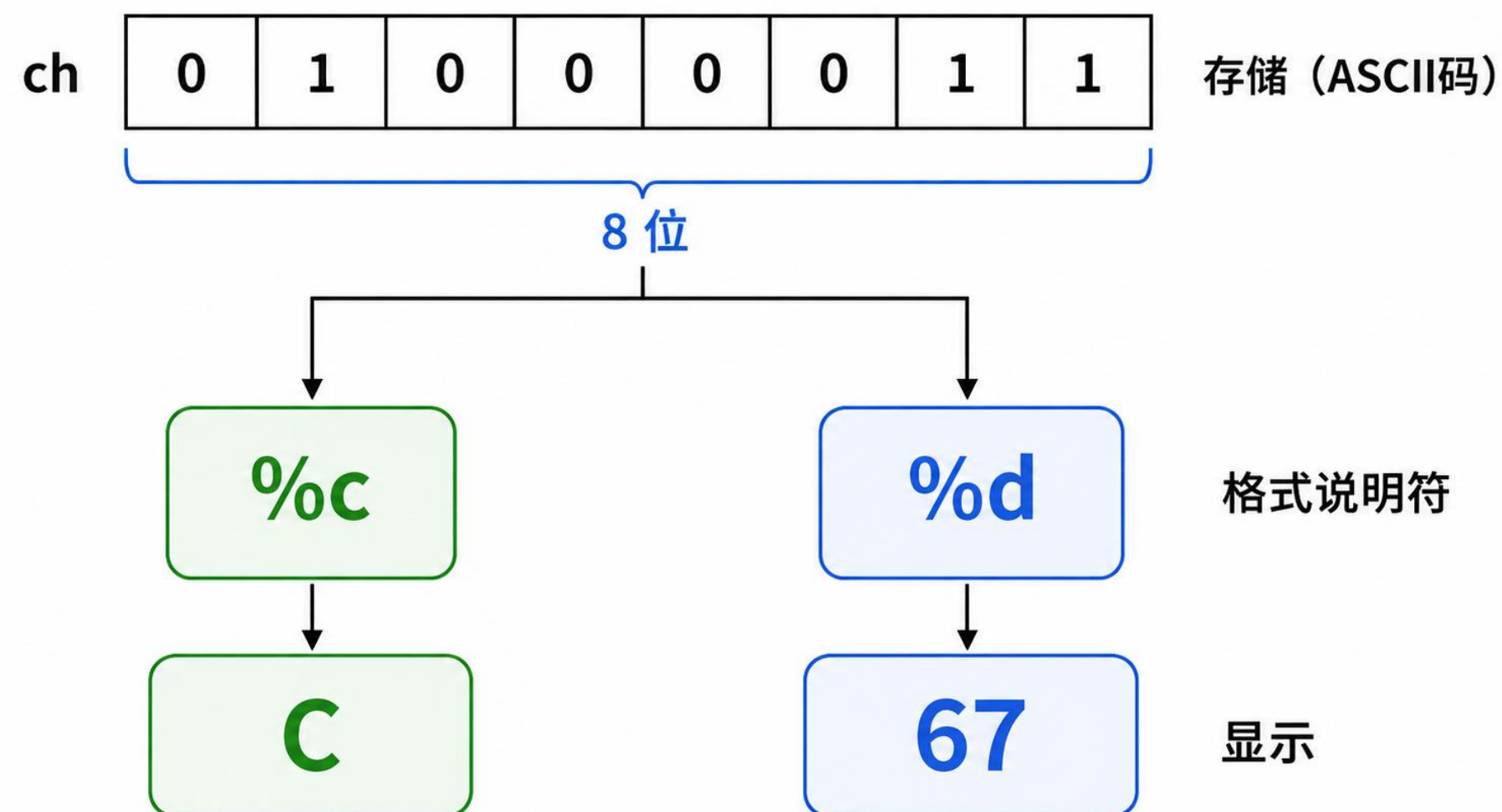
A screenshot of a Windows command prompt window. The title bar shows the path C:\WINDOWS\system32\cmd.exe. The command prompt displays the output '65535' on the first line and '请按任意键继续. . . ' on the second line, followed by a cursor.

字符类型 char

□ char 用于储存字符，如： 'C'、 'B'、 '1'、 '\n'

```
char ch = 'C';
```

□ 实质上 char 属于**整数类型**，占 **1** 字节，储存字符对应的 **ASCII 码**



字符类型 char

ASCII 值	控制字符	ASCII 值	控制字符	ASCII 值	控制字符	ASCII 值	控制字符
0	NUL	32	(space)	64	@	96	`
1	SOH	33	!	65	A	97	a
2	STX	34	"	66	B	98	b
3	ETX	35	#	67	C	99	c
4	EOT	36	\$	68	D	100	d
5	ENQ	37	%	69	E	101	e
6	ACK	38	&	70	F	102	f
7	BEL	39	.	71	G	103	g
8	BS	40	(	72	H	104	h
9	HT	41	)	73	I	105	i
10	LF	42	*	74	J	106	j
11	VT	43	+	75	K	107	k
12	FF	44	,	76	L	108	l
13	CR	45	-	77	M	109	m
14	SO	46	.	78	N	110	n
15	SI	47	/	79	O	111	o
16	DLE	48	0	80	P	112	p
17	DC1	49	1	81	Q	113	q
18	DC2	50	2	82	R	114	r
19	DC3	51	3	83	X	115	s
20	DC4	52	4	84	T	116	t
21	NAK	53	5	85	U	117	u
22	SYN	54	6	86	V	118	v
23	TE	55	7	87	W	119	w
24	CAN	56	8	88	X	120	x
25	EM	57	9	89	Y	121	y
26	SUB	58	:	90	Z	122	z
27	ESC	59	;	91	[	123	{
28	FS	60	<	92	\	124	
29	GS	61	=	93	]	125	}
30	RS	62	>	94	^	126	~
31	US	63	?	95	_	127	DEL

注意

- 字符 '1' 和整数 1 是不同的概念
- 字符 '1' 在内存中以 ASCII 码形式存储, 占 1 个字节

0 0 1 1 0 0 0 1

- 而整数 1 是以整数存储方式 (二进制) 存储的, 占 2 个或 4 个字节

[illegible]

$1+1 \neq '1'+'1'$

浮点类型

$3.14159 = 3.14159 \times 10^0 = 0.314159 \times 10^1 = 314.159 \times 10^{-2}$

□ 由于小数点位置可以浮动，所以实数的指数形式称为浮点数

浮点数据类型	类型形式	格式声明符	字节数	有效数字	取值范围 (绝对值)
单精度浮点型	float	%f	4	6 位	0 以及 $1.2 \times 10^{-38} \sim 3.4 \times 10^{38}$
双精度浮点型	double	%f %lf	8	15 位	0 以及 $2.3 \times 10^{-308} \sim 1.7 \times 10^{308}$
长双精度浮点型	long double	%Lf	与编译环境有关		

浮点类型

□ 考虑以下程序：

```
float a, b;  
b = 2e20 + 1.0;  
a = b - 2e20;  
printf("a = %f\n", a);
```

□ 任意不同实数间都有无穷多个数，浮点数通常只是实际值的**近似值**！

□ 浮点类型的储存方式



常见基本变量类型总结

类型	类型标识符	字节	数值范围	数值范围
整型	[signed] int	4	-2147483648 ~ +2147483647	$-2^{31} \sim +2^{31}-1$
无符号整型	unsigned [int]	4	0 ~ 4294967295	$0 \sim +2^{32}-1$
短整型	short [int]	2	-32768 ~ +32767	$-2^{15} \sim +2^{15}-1$
无符号短整型	unsigned short [int]	2	0 ~ 65535	$0 \sim +2^{16}-1$
长整型	long [int]	4	-2147483648 ~ +2147483647	$-2^{31} \sim +2^{31}-1$
无符号长整型	unsigned long [int]	4	0 ~ 4294967295	$0 \sim +2^{32}-1$
长长整型	long long [int]	8	-9223372036854775808 ~ 9223372036854775807	$-2^{63} \sim +2^{63}-1$
无符号长长整型	unsigned long long [int]	8	0 ~ 18446744073709551616	$0 \sim +2^{64}-1$
字符型	[signed] char	1	-128 ~ +127	$-2^7 \sim +2^7-1$
无符号字符型	unsigned char	1	0 ~ +255	$0 \sim +2^8-1$
单精度型	float	4	$-3.4 \times 10^{38} \sim 3.4 \times 10^{38}$	$-3.4 \times 10^{38} \sim 3.4 \times 10^{38}$
双精度型	double	8	$-1.7 \times 10^{308} \sim 1.7 \times 10^{308}$	$-1.7 \times 10^{308} \sim 1.7 \times 10^{308}$
长双精度型	long double	8	$-1.7 \times 10^{308} \sim 1.7 \times 10^{308}$	$-1.7 \times 10^{308} \sim 1.7 \times 10^{308}$

如何知道类型大小? sizeof()

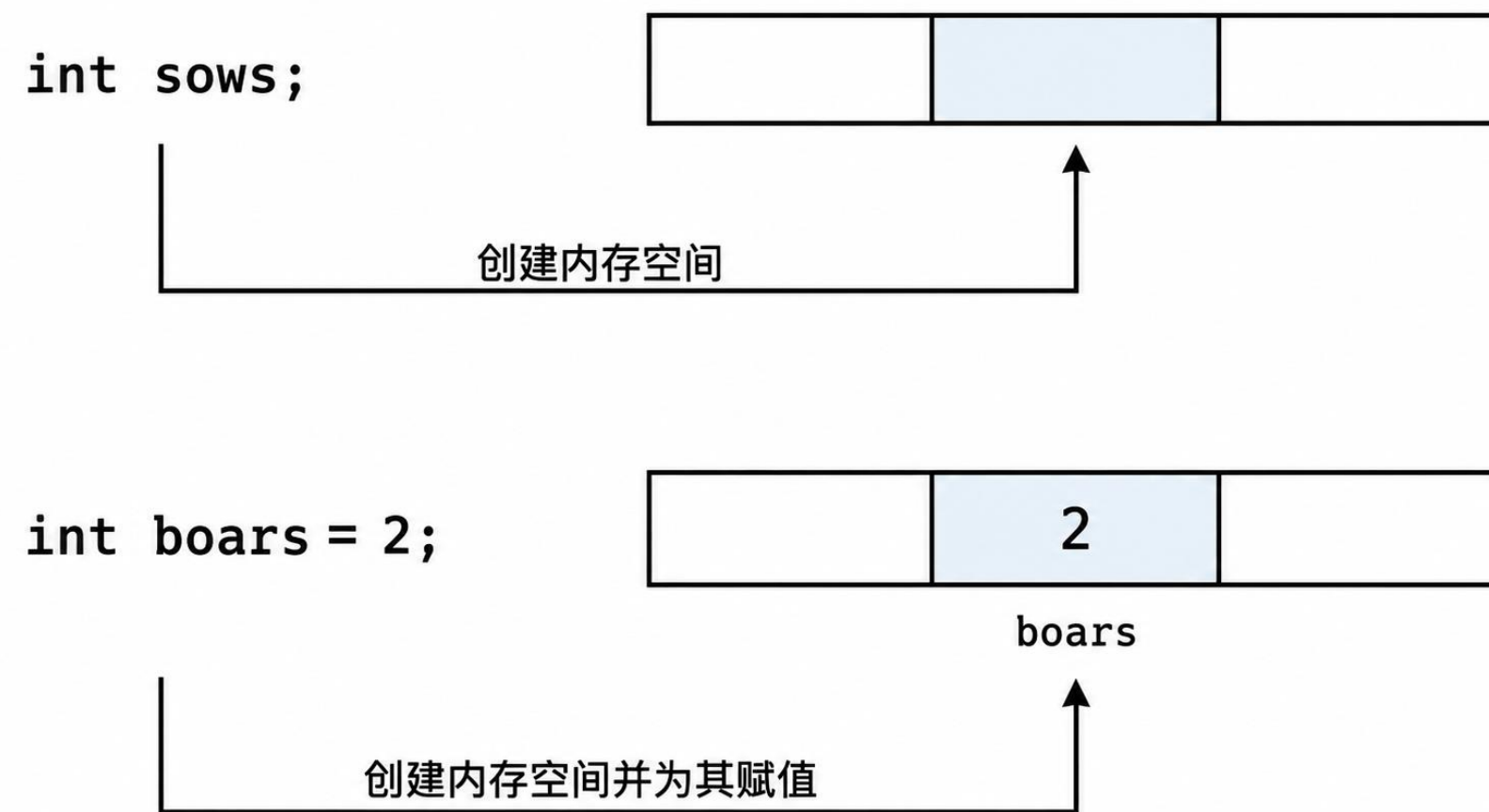
变量的声明

- 变量声明就是向程序表明变量的**类型和名称**，并**创建、标记存储空间**
- 具体语法：**<类型> <变量名>;**
 - <类型> 代表有效的C数据类型，<变量名> 可以由一个或多个标识符名称组成，多个标识符之间用 “,” 分隔
- 几个有效的声明：

```
/*define vars and constants*/  
double radius, area;  
int i,j;  
char c;
```
- 通常使用**熟悉的单词作为变量名称**，例如radius、height等

变量初始化

- 变量可以在声明时初始化 (指定初值)
- 初始化器由一个赋值运算符，后跟一个 (常量) 表达式组成



- 注意，这样对于a、b的初始化是不允许的

```
int a = b = 3;
```

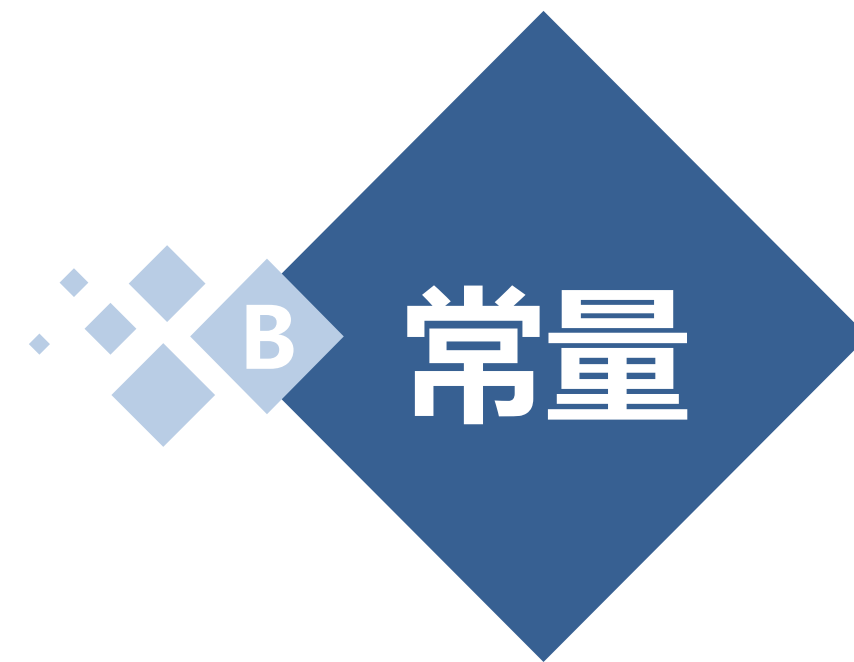
变量使用与赋值

- 定义或声明后的变量，可用“=”赋值，称为赋值运算符
- 定义的变量可以在表达式中使用，如

```
/*define vars and constants*/  
double radius = 5;  
double area = 0;  
  
area = radius * radius * 3.14;
```

重新赋值

在计算机高级语言中，数据的两种表现形式



常量

- 常量是固定值，在程序执行期间不会改变
- 主要包括**整型常量**、**浮点常量**、**字符常量**、字符串常量与符号常量

23——整型常量

3.14159——浮点型常量

'n'——字符常量

整型常量

- 不带小数点的数值为整数常量
- 可以是十进制、八进制或十六进制，用**整数前缀**表示非十进制，例如
 - 八进制：前缀 **0**，如 071 (十进制 57)
 - 十六进制：前缀 **0x**，如 0xFF (十进制 255)
- 用**整数后缀**表示**无符号整数 (u/U)** 以及**长整型整数 (l/L)**

212	/* 合法的 */
215u	/* 合法的 */
0xFEeL	/* 合法的 */
078	/* 非法的: 8 不是八进制的数字 */
032UU	/* 非法的: 不能重复后缀 */

```
int x = 100;
printf("%d, %o, %x\n", x, x, x);
```

浮点常量

□ 以**小数形式**或**指数形式**出现的实数是浮点型常量

1. 小数形式

必须包含小数点；可以有整数部分、小数部分，或同时包含两者

3.14 → 同时有整数部分和小数部分

12. → 只有整数部分

.75 → 只有小数部分

-8. → 负整数，只有整数部分

-0.25 → 负数，同时有整数和小数部分

2. 指数形式

E/e 前面是尾数部分，必须有数；
E/e 后面是指数部分，必须是整数

2e3 → $= 2 \times 10^3$

3.5E-2 → $= 3.5 \times 10^{-2}$

.8e+1 → $= 0.8 \times 10^1$

常见错误

× **e3** → 尾数部分不能为空

× **3.2e1.5** → 指数部分必须是整数

□ C编译系统把浮点常量默认为 double 类型，分配8字节

字符常量

□ 字符常量是用单引号括起来的一个字符

```
'A'  'b'  '3'  '+'  '\n'
```

□ 注意区分：字符串常量是用双引号括起来的一个或多个字符序列

```
"A"  "Hello world"  "123"
```

字符常量

转义字符	意义	ASCII码值
\a	响铃 (BEL)	007
\b	退格 (BS) , 将当前位置移到前一系列	008
\f	换页 (FF) , 将当前位置移到下页开头	012
\n	换行 (LF) , 将当前位置移到下一行开头	010
\r	回车 (CR) , 将当前位置移到本行开头	013
\t	水平制表 (HT) , 跳到下一个 TAB 位置	009
\v	垂直制表 (VT)	011
\\	代表一个反斜线字符 “\”	092
\'	代表一个单引号 (撇号) 字符	039
\"	代表一个双引号字符	034
\0	空字符 (NULL)	000
\ddd	1 ~ 3 位八进制数所代表的任意字符	三位八进制
\xhh	1 ~ 2 位十六进制数所代表的任意字符	二位十六进制

变量与常量

1. 当把常量赋值给不同类型变量时，会发生**隐式类型转换**，如

```
int num = 1.99;  
float pi = 3.1415926536;
```

这种转换可能**损失精度**

2. 可以在常量的末尾加专用**后缀**，强制指定常量的类型

```
float a = 3.14159f;    //把3.14159按float常量处理  
long double a = 1.23L; //把1.23作为long double型处理
```

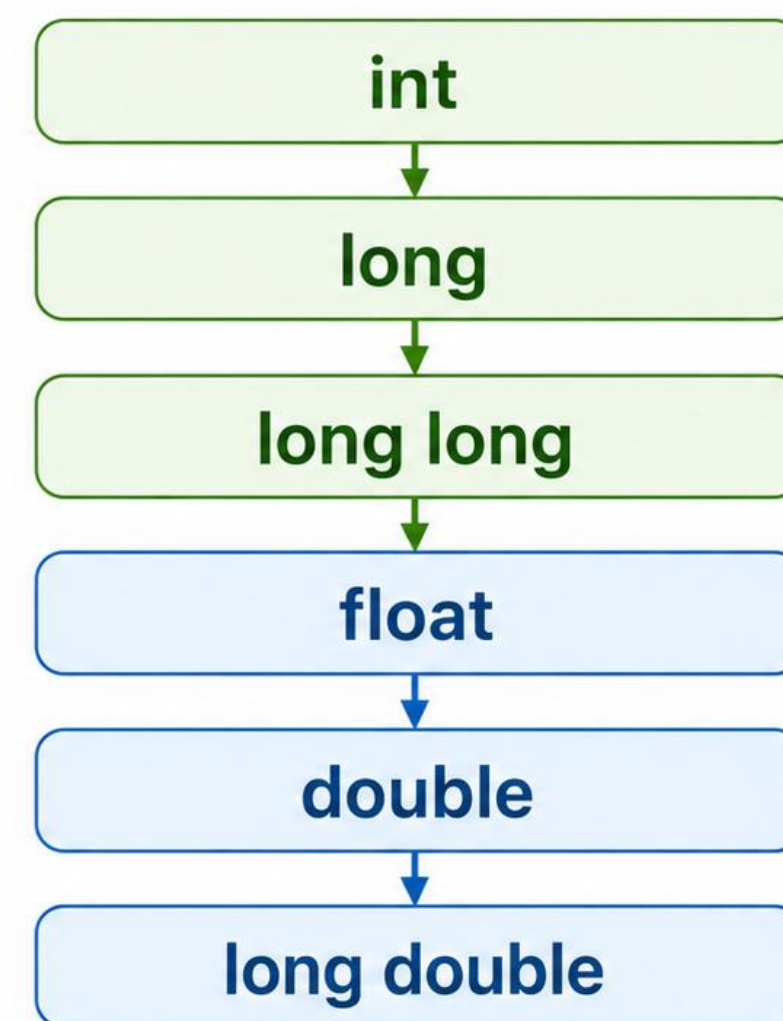
练习

- 某服务器平均每秒处理1000次请求，假设服务器连续运行30天，请编写程序，计算并输出这30天服务器一共处理了多少次请求

```
total = 2592000000
```

```
#include <stdio.h>
int main(void) {
    long long total;
    total = 30LL * 24 * 60 * 60 * 1000;
    printf("total = %lld\n", total);
    return 0;
}
```

运算涉及**不同类型**数据时，会先进行**类型转换**，再完成运算



字符串常量与符号常量

□ 字符串常量

- 用双引号括起来的一串字符，如 "Hello"、"abc"
- 常用于输出提示信息

```
printf("Hello, world!\n");
```

□ 符号常量

- 用标识符代表一个常量，通常用 **#define** 定义
- 便于理解和统一修改

```
#define PI 3.14159  
...  
area = PI * r * r;
```

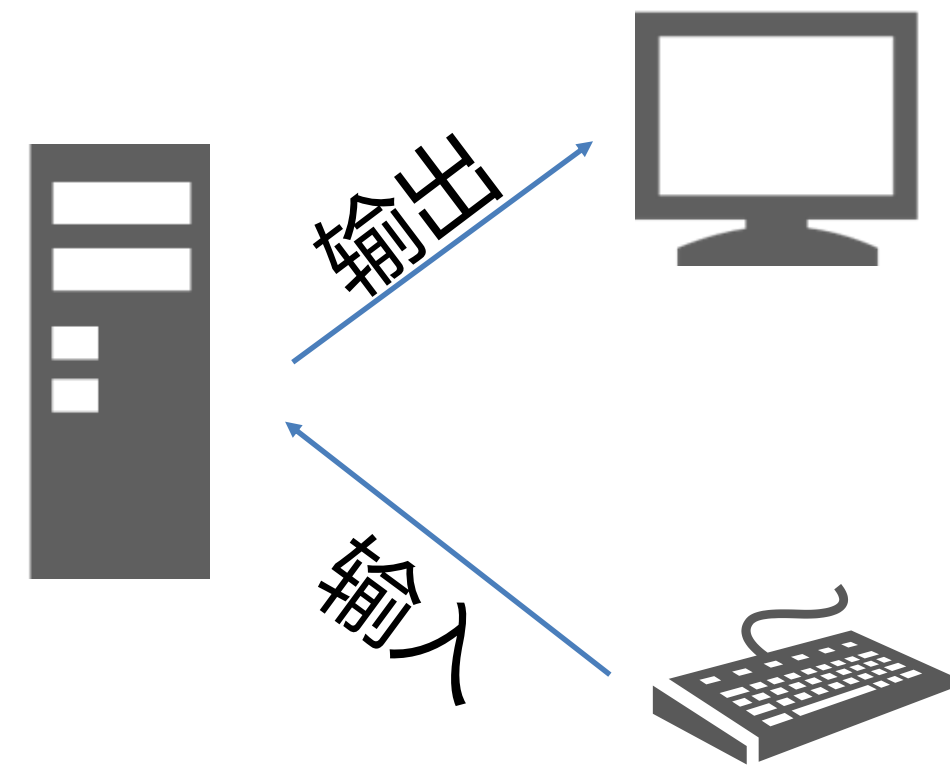
本节内容

□ 如何写出能处理数据的程序？



输入与输出

- 输入输出是以计算机主机为主体而言的
- **输入**：从外部设备读入数据到内存变量
 - 默认的输入设备称为**标准输入**，一般是**键盘**
 - **常用标准输入函数**：**scanf, getchar**
- **输出**：把内存中变量或表达式计算结果读出到显示器、打印机或磁盘文件等外部设备
 - 默认的输出设备称为**标准输出**，一般是**控制台**
 - **常用标准输出函数**：**printf, putchar**
- 标准输入、输出函数在 **stdio.h** 中声明

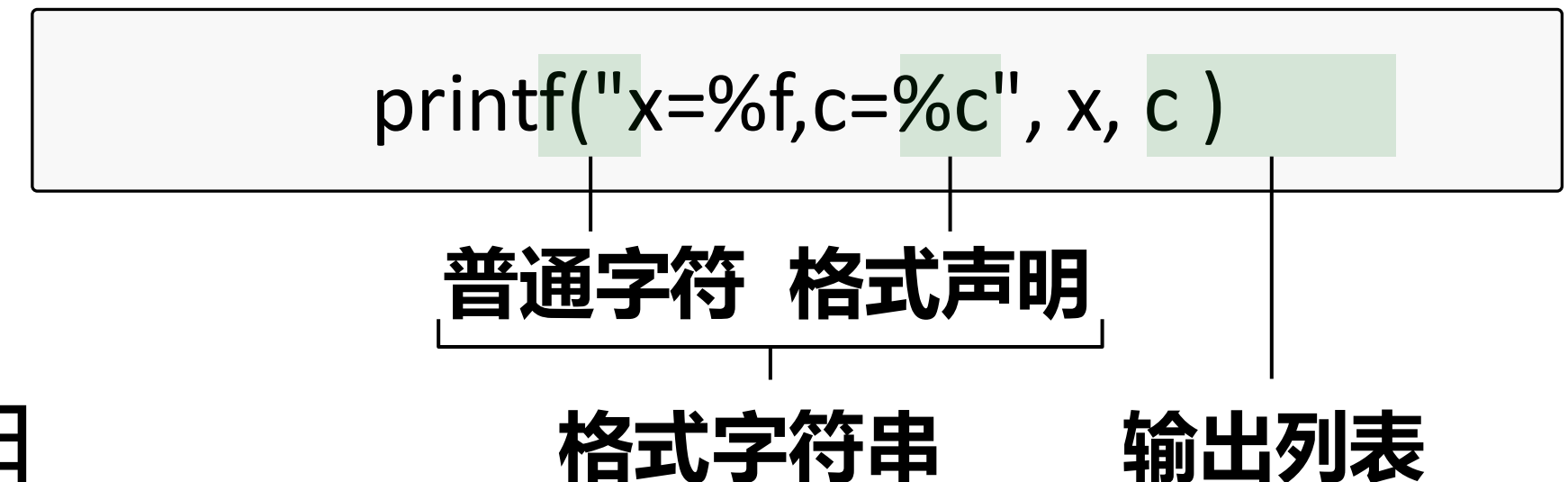


```
#include <stdio.h>
```

printf函数

- 用来向标准输出设备输出若干任意类型的数据

printf(格式字符串,输出表列)



- 格式字符串包括普通字符和格式声明
 - 普通字符在输出时原样输出的字符
 - 格式声明由“%”和格式字符组成，作用是将输出的数据翻译为指定格式后输出，与输出列表中项一一对应
- 输出表列是程序需要输出的数据，可以是常量、变量或表达式

printf函数——格式声明

格式声明：**%** **附加字符** **格式字符**

```
printf("x=%f,c=%c", x, c )
```

□ 格式声明符需要与输出对象保持一致：

1. 格式声明符的个数与参数**个数一致**
2. 格式声明符的类型与参数**类型一致**

printf函数——格式声明

格式字符	说 明	附加字符	说明
d, i	以带符号的十进制形式输出整数 (正数不输出符号)	-	在指定宽度内左对齐，默认右对齐
o	以八进制无符号形式输出整数 (不输出前导符0)	+	正数前显示+，负数前显示-
x, X	以十六进制无符号形式输出整数 (不输出前导符0x)，用x则输出十六进制数的a ~ f时以小写形式输出，用X时，则以大写字母输出	空格	正数前留一个空格；与+同时使用时，+优先
u	以无符号十进制形式输出整数	0	宽度不足时在左侧补0；与-同时使用时失效
c	以字符形式输出，只输出一个字符	#	输出八进制、十六进制前缀0、0x或0X；浮点数保留小数
s	输出字符串	m	指定最小输出宽度；数据超过m时按实际宽度输出
f	以小数形式输出单、双精度数，隐含输出6位小数	.n	指定精度：f、e保留n位小数，g保留n位有效数字，s最多输出n个字符
e, E	以指数形式输出实数，用e时指数以“e”表示 (如1.2e+02)，用E时指数以“E”表示 (如1.2E+02)	h、hh	分别用于short和signed/unsigned char，如 %hd、%hhu
g, G	选用%f或%e格式中输出宽度较短的一种格式，不输出无意义的0。用G时，若以指数形式输出则指数以大写表示	l、ll	分别用于 long 和 long long，如 %ld、%lld

示例

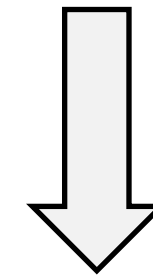
```
/*format testing*/
#include<stdio.h>

int main(void) {

    printf("%lf\n", 5.0);
    printf("%.2lf\n", 5.0);
    printf("%10lf\n", 5.0);
    printf("%10.2lf\n", 5.0);
    printf("%-10.2lf\n", 5.0);
    printf("%+10.2lf\n", 5.0);
    printf("% .2lf\n", 5.0);
    printf("% .2lf\n", -5.0);
    printf("%010.2lf\n", 5.0);

    return 0;
}
```

printf 格式声明 : % 附加字符 格式字符

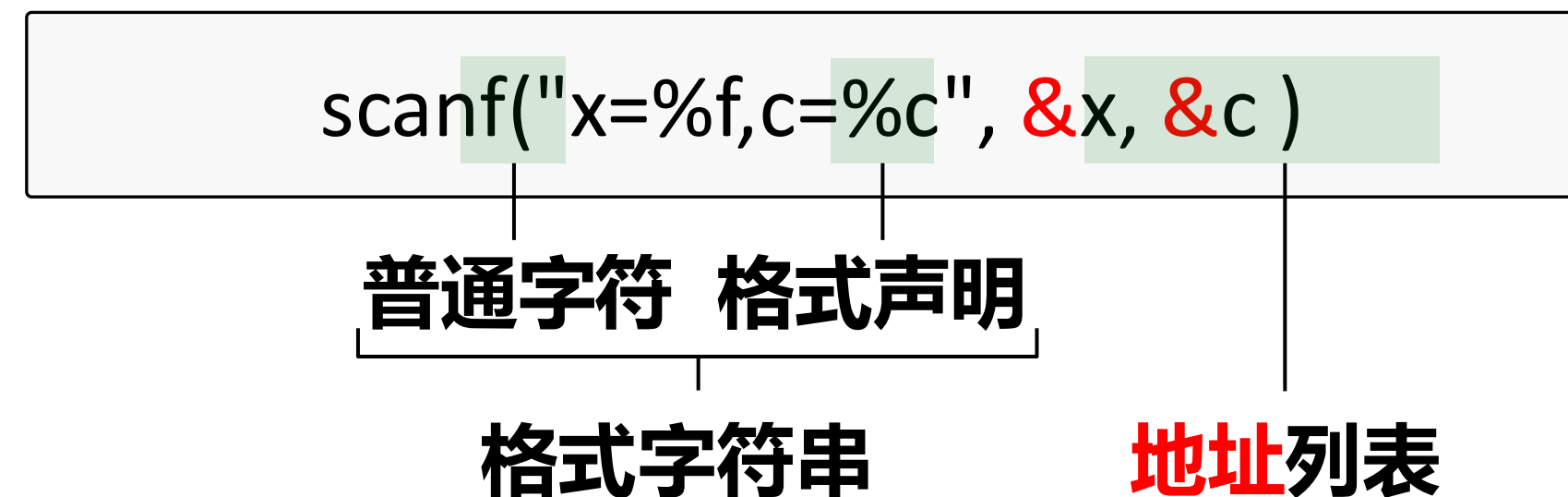


%[标志][宽度][.精度][长度修饰符]格式字符

scanf函数

- 用来从标准输入设备读入数据

scanf(格式字符串,地址表列)



- 格式字符串含义同 printf 函数
 - 格式化字符中普通字符必须与输入的一样
 - 变量数量和类型必须与格式化字符串需要的类型一致
- 地址表列是指变量的地址组成的列表

scanf函数——格式声明

格式字符	说明
d, i	d: 以十进制形式输入有符号整数 i: 输入有符号整数, 并根据前缀自动判断进制 (普通数字为十进制, 0开头为八进制, 0x开头为十六进制)
o	以八进制形式输入无符号整数
x, X	以十六进制形式输入无符号整数, 可带前缀0x或0X; x与X作用相同
u	以十进制形式输入无符号整数
c	输入一个字符, 包括空格、制表符和换行符; 不会自动跳过空白字符
s	输入字符串: 跳过开头的空白字符, 读取到下一个空白字符为止, 并在末尾自动添加'\0'
f	输入float型实数 , 可以使用小数形式或指数形式输入
e, E	作用与f相同, 均可输入实数; e与E作用相同
g, G	作用与f、e相同, 均可输入实数; g与G作用相同

附加字符	说明
空格	格式串中的空白表示跳过任意数量的空白字符
m	指定最多读取的字符数, 例如%5d最多读取5个字符
*	读取输入, 但不保存到变量中 , 例如%*d表示读入并丢弃一个整数
h、hh	分别用于short和signed/unsigned char, 如 %hd、%hhu
l、ll	l 用于long, %lf表示double ; ll用于long long, 如%lld
L	用于long double, 如%Lf、%Le、%Lg

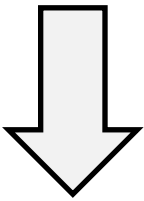
- 1. scanf 中 **%f** 和 **%lf** 作用不同
- 2. 用 %c 时, **空格字符**和**转义字符**都作为有效字符输入
- 3. 在输入数值数据时, 如输入**空格**、**回车**等**不属于数值的字符**, 认为该数据结束在输入结束

scanf函数——格式声明

格式字符	说明
d, i	d: 以十进制形式输入有符号整数 i: 输入有符号整数, 并根据前缀自动判断进制 (普通数字为十进制, 0开头为八进制, 0x开头为十六进制)
o	以八进制形式输入无符号整数
x, X	以十六进制形式输入无符号整数, 可带前缀0x或0X; x与X作用相同
u	以十进制形式输入无符号整数
c	输入一个字符, 包括空格、制表符和换行符; 不会自动跳过空白字符
s	输入字符串: 跳过开头的空白字符, 读取到下一个空白字符为止, 并在末尾自动添加'\0'
f	输入float型实数, 可以使用小数形式或指数形式输入
e, E	作用与f相同, 均可输入实数; e与E作用相同
g, G	作用与f、e相同, 均可输入实数; g与G作用相同

附加字符	说明
空格	格式串中的空白表示跳过任意数量的空白字符
m	指定最多读取的字符数, 例如%5d最多读取5个字符
*	读取输入, 但不保存到变量中, 例如%*d表示读入并丢弃一个整数
h、hh	分别用于short和signed/unsigned char, 如 %hd、%hhu
l、ll	l 用于long, %lf表示double; ll用于long long, 如%lld
L	用于long double, 如%Lf、%Le、%Lg

scanf 格式声明 : % 附加字符 格式字符



%[*][宽度][长度修饰符]格式字符

练习

□ 编写程序，使用 scanf 从一行输入中依次读取：

1. 一个不超过3位的整数 number

2. 一个 float 型实数 price

3. 一个 double 型实数 score

4. 一个字符 grade

5. 一个整数，但不保存该整数

并使用 printf 打印这四个变量

练习

```
#include <stdio.h>

int main(void) {
    int number;
    float price;
    double score;
    char grade;

    scanf("%d %f %lf %c %*d", &number, &price, &score, &grade);

    printf("number = %d\n", number);
    printf("price = %.2f\n", price);
    printf("score = %.2f\n", score);
    printf("grade = %c\n", grade);

    return 0;
}
```



字符输入输出函数

❑ getchar 函数：从标准输入读入一个字符

```
ch = getchar();
```

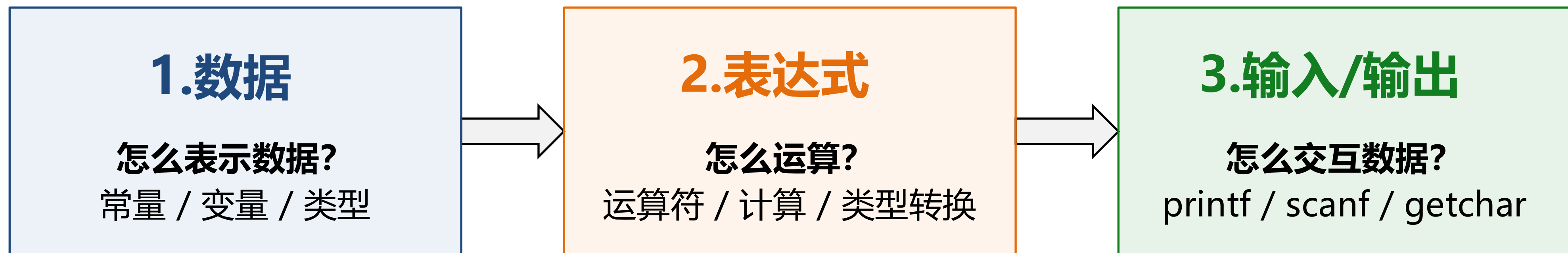
❑ putchar 函数：向标准输出打印一个字符

```
putchar(ch);
```

❑ 例：从键盘输入 BOY 3个字符，然后把它们输出到屏幕

```
#include <stdio.h>
int main(void){
    char a,b,c;           //定义字符变量a,b,c
    a=getchar();          //从键盘输入一个字符，送给字符变量a
    b=getchar();          //从键盘输入一个字符，送给字符变量b
    c=getchar();          //从键盘输入一个字符，送给字符变量c
    putchar(a);           //将变量a的值输出
    putchar(b);           //将变量b的值输出
    putchar(c);           //将变量c的值输出
    putchar('\n');        //换行
}
```

总结



作业

- 阅读《C程序设计》第2-3章
- 完成 PTA 本周习题

补充

进位制

□ 642 的十进制表示法为：

$$6 \times 10^2 + 4 \times 10^1 + 2 \times 10^0 = (642)_{10}$$

表示基为10

指数表示在数值
中的位置

进位制

□ 642的十进制表示:

$$6 \times 10^2 + 4 \times 10^1 + 2 \times 10^0 = (642)_{10}$$

□ **R进制**的公式表示:

$$d_n \times R^{n-1} + d_{n-1} \times R^{n-2} + \cdots + d_2 \times R + d_1 = (d_n d_{n-1} \cdots d_1)_R$$

$$0 \leq d_i \leq R - 1, \quad d_n \neq 0$$

进位制

□ **十进制**的基是10，有10个数字：

■ 0, 1, 2, 3, 4, 5, 6, 7, 8, 9

□ **二进制**的基是2，有2个数字：

■ 0, 1

□ **八进制**的基是8，有8个数字：

■ 0, 1, 2, 3, 4, 5, 6, 7

□ **十六进制**的基是16，有16个数字：

■ 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F

二进制与十六进制、八进制的关系

□ 二进制与八进制按3位一组对应，与十六进制按4位一组对应

二进制	八进制	十六进制
0000	0	0
0001	1	1
0010	2	2
0011	3	3
0100	4	4
0101	5	5
0110	6	6
0111	7	7

二进制	十六进制
1000	8
1001	9
1010	A
1011	B
1100	C
1101	D
1110	E
1111	F

$$\begin{aligned} (DEF)_{16} &= (1101\ 1110\ 1111)_2 \\ &= (110\ 111\ 101\ 111)_2 \\ &= (6757)_8 \end{aligned}$$

练习

□ 计算以下数的二进制、八进制、十六进制值

$(143)_{10}$

□ 二进制：10001111

□ 八进制：217

□ 十六进制：8F

将十六进制与八进制数赋值给变量

```
/*hex-oct-int*/  
#include <stdio.h>  
  
int main(void) {  
    unsigned int i = 0xDEF;  
    unsigned int j = 06757;  
    printf("i in Decimal is %u\n",i);  
    printf("j in Decimal is %u\n",j);  
    return 0;  
}
```

0x 开头表示是 十六进制 的数值

0 开头表示是 八进制 的数值

溢出 (Overflow)

```
/*overflow*/
#include <stdio.h>

int main(void) {
    char k = 127; //0x7F;
    char m = k + 10;
    char n = k + k + 10;
    printf("%d, %d, %d\n", k+10, m, n);
}
```

输出是：

137, -119, 8

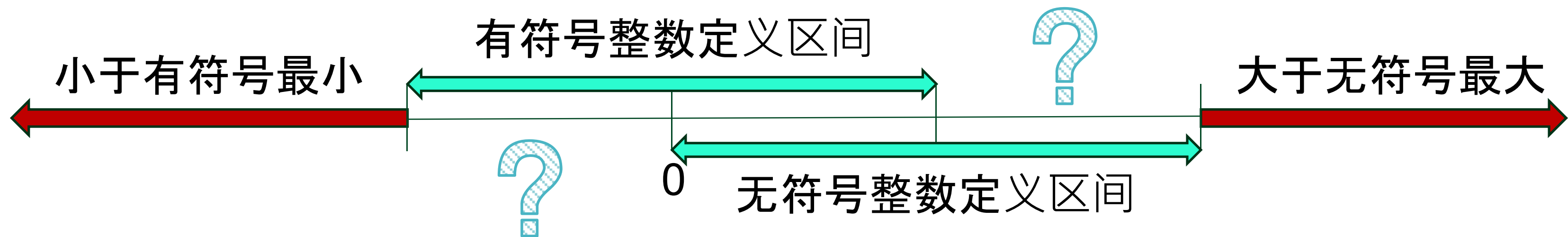
- ❑ 表达式 $k+10$ 、 $k+k+10$ 在运算时，先按int计算
- ❑ 当结果赋值给char变量时，只保留低8位，高位被截断
- ❑ $k+10=137=0x89$ ，作为int是137；赋给8位有符号char后，按补码解释为-119

1	0	0	0	1	0	0	1
---	---	---	---	---	---	---	---
- ❑ $k+k+10=264=0x108$ ，赋给n时只保留低8位0x08，即8

1	0	0	0	0	1	0	0	0
---	---	---	---	---	---	---	---	---

溢出 (Overflow)

- ❑ **整数溢出**：赋予变量的值超出变量类型定义的范围
- ❑ C语言语法层面可对常数赋值进行溢出检查，例如：char i = 129;



- ❑ 表达式赋值则按转换规则，常不做做任何检查
- ❑ **C语言开发者有义务保证不发生溢出！**

历史上著名的溢出缺陷 (BUG)

- ❑ 1996年6月4日，欧洲 Ariane 5火箭首次发射，升空约37秒后，火箭因飞行控制异常偏离航线并解体爆炸，造成约6亿美元损失
- ❑ 错误原因：程序将一个**64位浮点数转换为16位有符号整数**时，数值超出了16位整数的表示范围，引发操作数错误



浮点数与格式化

```
/*floating format*/
```

```
#include <stdio.h>
```

```
int main(void) {
```

```
    double x,y;
```

```
    x = -2.263;
```

```
    y = 13.367;
```

```
    double z = x + y;
```

```
    // 详细说明见: https://zh.cppreference.com/w/c/io/fprintf
```

```
    printf("%-8.2f add %08.2f equal %.4e\n",x,y,z);
```

```
    return 0;
```

```
}
```

`double` 表示浮点类型，这里定义了变量 `x` 和 `y`

变量赋值

精度4位，科学计算法输出

用0补齐，宽度8，精度2位，浮点数输出

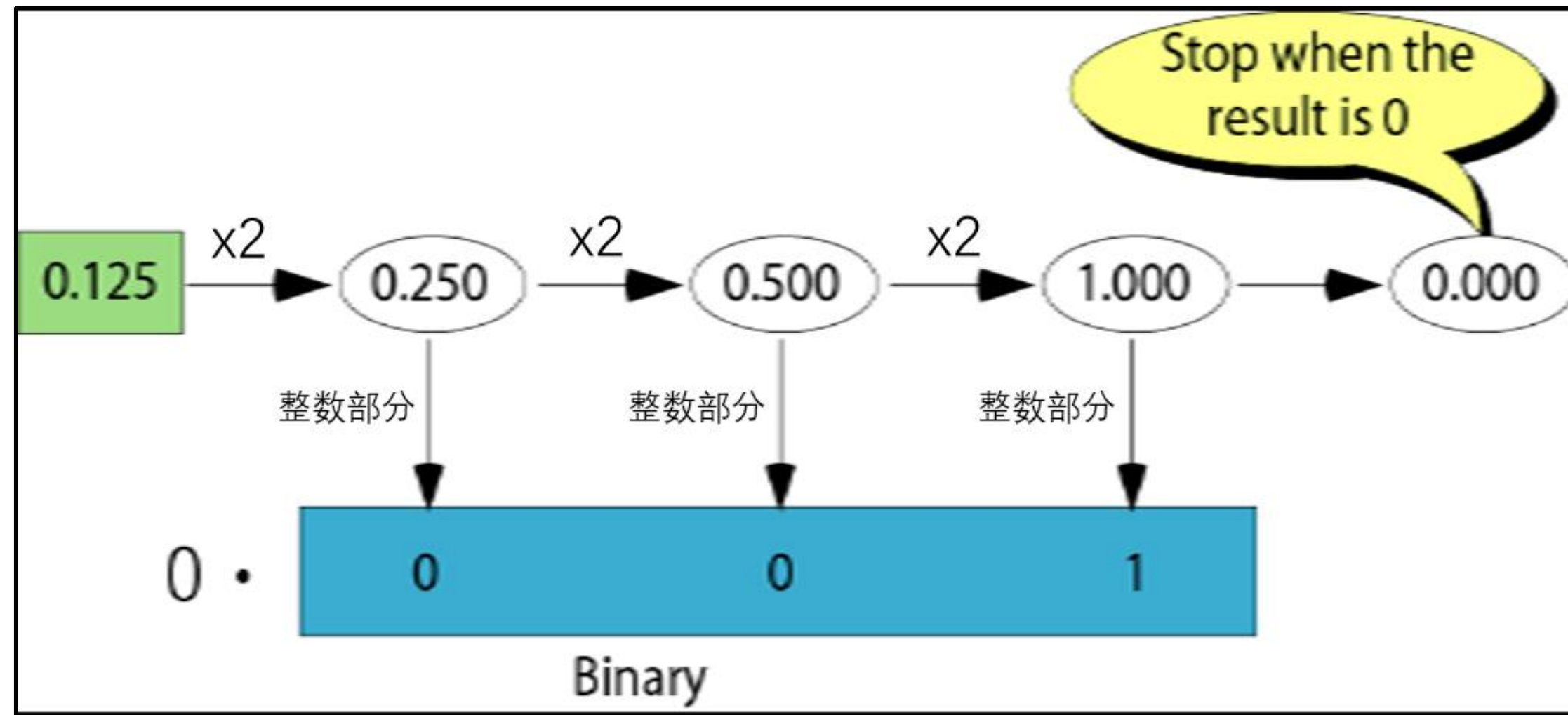
左对齐，宽度8，精度2位，浮点数输出

输出是：

```
-2.26  add 00013.37 equal 1.1104e+001
```

小数转二进制*

□ 基本原理与方法:



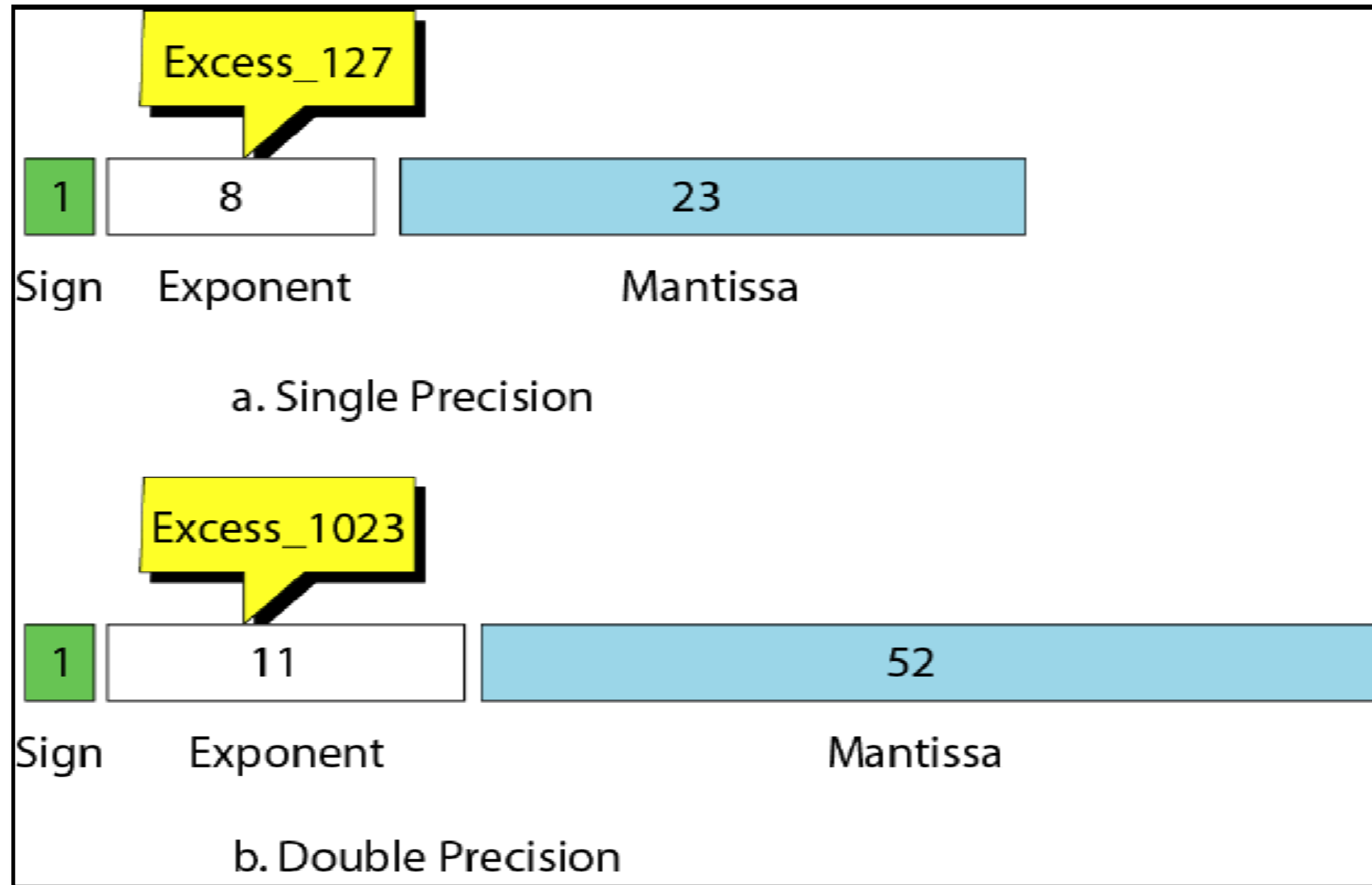
0.15
x2 0.30 -- 0
x2 0.60 -- 0
x2 1.20 -- 1 *
x2 0.40 -- 0 *
x2 0.80 -- 0 *
x2 1.60 -- 1 *
x2 1.20 -- 1 ...

$0.15 = (0.00100110011001...)_{2}$

□ 只要用二进制表示小数部分，绝大多数十进制数转二进制必然产生舍入误差 (精度误差)

IEEE 754-2008 标准与浮点数内存格式*

单精度数与双精度数在内存中的存储格式:



实数转为二进制表示：

1. 把实数的整数和小数部分分别转为二进制
2. 表示成二进制科学计数法
3. 按IEEE规范，计算符号位，指数部分，小数部分

例如：0.15

= 1.0011 0011 0011 0011 0011

001[1] * 2^{-3}

注意：四舍五入

float x = 0.15f;

按IEEE 745规范：

Sign 0

Exp -3 + 127 即 0x7C = 0111 1100

Manti 0011,0011,0011,0011,0011,010

结果：

0011 1110 0001 1001

1001 1001 1001 1010

x 的二进制存储是 0x3E19999A

标识符命名风格

- 使用**有意义、易读**的标识符
- **Unix like 风格**：用大或小写字母，单词间用下划线分割
 - 大写常用于常量，例如：PI, E, INT_MAX
 - 小写常用于局部变量、函数等，例如：printf, x, length, my_age
- **驼峰风格**：首字母大写的单词连接一起
 - 大驼峰风格：常用于全局类型名、结构名、函数等，如：Point, ContextMenu
 - 小驼峰风格：常用于局部的变量、函数等，如：textColor, numberOfYears
- **匈牙利风格**：用一个前缀表示数据类型
 - 整型变量 intCount、浮点变量 floatScore

谢谢